

Assignment 2 - bonus point

Min Zhu

April 25, 2026

Exercise 5.1

After completing the 2-layer network and the coarse-fine search of λ , further steps are taken to improve the performance of the network. For reference, the 2-layer network with best $\lambda = 5.27 \times 10^{-4}$ achieves a test accuracy of 52.01% under the setting `{eta_min=1e-5, eta_max = 1e-1, n_s =980 , n_batch = 100, n_cycle =3}`.

a. Add Hidden Nodes

To improve the performance of the network, one straightforward approach is to increase the number of hidden nodes.

In this experiment, all other hyperparameters and training settings are kept fixed, while only the number of hidden nodes m is varied. The networks trained under different m are then evaluated on the test set, and the corresponding test accuracies are reported in Table 1.

It can be observed that as the number of hidden nodes increases, the test accuracy improves consistently. The best performance is achieved when $m = 800$, where the test accuracy reaches **56.72%**.

Table 1: Optimal test accuracy under different m

m (Number of hidden nodes)	Test Accuracy	Improvement
50	0.5201	-
100	0.5355	0.0154
200	0.5539	0.0338
400	0.5574	0.0373
800	0.5672	0.0471

b. Dropout

As the number of hidden nodes increases, the model becomes more prone to overfitting. To address this, dropout is employed as an effective regularization technique.

Considering the trade-off between test accuracy and computational efficiency, dropout is implemented for the model with $m = 400$ hidden nodes. p_{keep} are chosen from 0.5-1, corresponding to different dropout strengths. The experimental results are presented in Table 2.

The results indicate that in this case a relatively weak strength of dropout contributes to improved model performance. The optimal outcome is obtained at $p_{keep} = 0.8$, achieving a test accuracy of **57.01%**.

Table 2: Dropout effect when $m=400$

p_{keep}	p_{drop}	Test Accuracy	Improvement
1.0	0.0	0.5574	-
0.9	0.1	0.5695	0.0121
0.8	0.2	0.5701	0.0127
0.7	0.3	0.5685	0.0111
0.6	0.4	0.5627	0.0153
0.5	0.5	0.5531	-0.0043

c. Data Augmentation

To enhance the model's ability to recognize variations in image appearance, I use data augmentation method. Specifically, random transformations including image translation and horizontal flipping are applied to the training data, allowing the model to learn more robust and invariant features.

The visualization images before and after random translation are shown in Fig.1 to show Correct implementation.

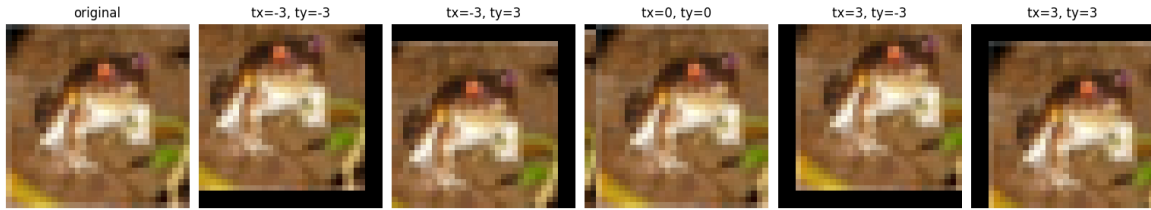


Figure 1: Visualization of random translation

To clearly demonstrate the effectiveness of this particular approach, the basic model is used as a baseline, with 50 hidden nodes and no dropout applied. All other training parameters are kept unchanged to ensure a fair comparison. Then I try this method in different values of m and compare the results with those in Table 1.

Table 3: Results with and without data augmentation

Number of hidden nodes	Test Accuracy without DA	Test Accuracy with DA	Improvement
50	52.01%	52.84%	0.83%
100	53.55%	53.90%	0.45%
200	55.39%	56.33%	0.94%
400	55.74 %	56.69%	0.95%

After applying data augmentation, the test accuracy shows a slight improvement across different values of m .

Exercise 5.2 Semi-extension Testing

Pre-train for Data Augmentation

As suggested in the assignment, random translations can be computationally expensive, especially when repeated extensively during hyperparameter search. To improve efficiency, I preprocessed all images by applying translations along both the x and y axes within the range $[-3, 3]$, resulting in a total of 49 possible combinations per image. These augmented samples were generated and stored in advance. During subsequent model training, augmented data can then be efficiently accessed through random selection, significantly reducing computational overhead while maintaining the benefits of data augmentation.

Coarse-to-fine Search

In this section, semi-extensive experiments are conducted to explore the performance limits of a two-layer network.

Building upon the implementation of data augmentation and an increased number of hidden nodes (up to 1024), a coarse-to-fine parameter search is performed to identify the optimal combination of hyperparameters, including $\eta_{\max}$, λ , P_{dropout} . In the coarse search stage, a random search strategy is employed over 30 trials. The hyperparameters are sampled from the following ranges:

$$\lambda \in [10^{-6}, 10^{-2}], \eta_{\max} \in [10^{-2}, 5 \times 10^{-1}], P_{\text{dropout}} \in \{0.5, 0.6, 0.7, 0.8, 0.9\}.$$

Top five of Sorted coarse search result is as following:

rank	trial_id	lambda	eta_max	p_dropout	best_val_acc
1	22	0.000504	0.071213	0.9	0.551
2	9	1.13E-06	0.045847	0.8	0.545
3	0	5.20E-06	0.088151	0.6	0.544
4	5	0.001025	0.057669	0.8	0.544
5	18	9.39E-05	0.071907	0.8	0.543

Figure 2: Coarse Search Result

The fine search further refines the top five combinations obtained from the coarse search, small-range are set around their parameter values. The results are summarized below, and the optimal combinations are:

$$\lambda = 1.68 \times 10^{-4}, \eta_{\max} = 4.906 \times 10^{-2}, P_{\text{dropout}} = 0.865.$$

rank	trial_id	lambda	eta_max	p_dropout	best_val_acc
1	31	0.000168	0.04906	0.865058	0.58
2	34	0.000637	0.080127	0.962473	0.577
3	39	1.47E-06	0.038333	0.842566	0.575
4	50	0.001449	0.086072	0.854756	0.574
5	40	4.16E-07	0.055816	0.857133	0.573

Figure 3: Fine Search Result

Run Long Train

Based on the optimal parameters found above, more training cycles are adopted to increase performance, when n_{cycle} is set to be 5, the corresponding loss and accuracy on the training and validation data are shown in Fig.4, and the test accuracy is 58.33%.

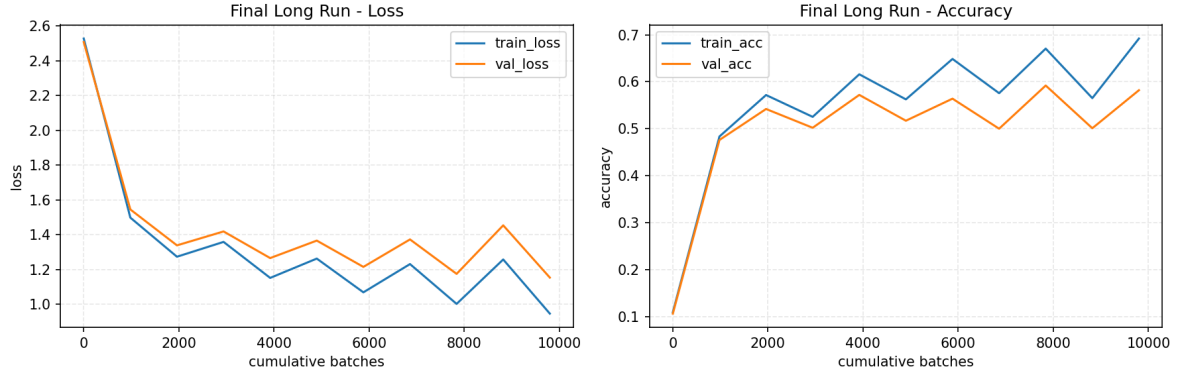


Figure 4: Long Train($n_{cycle} = 5$)

Annealing Learning Rate

Furthermore, an annealing learning rate strategy is added into the model. Specifically, the total training process consists of 10 cycles: the initial six cycles use the standard learning rate, while the remaining four cycles adopt a reduced learning rate. This leads to a learning curve as shown below:



Figure 5: Long Train($n_{cycle} = 10$) + Annealing learning rate

Finally, the best performance I obtain with a wide 2-layer network is 59.38%.

```
[Phase 2] annealing training
Test accuracy under searched best parameters: 0.593800
```

Figure 6: Best Performance

Exercise 5.3 Adam Optimizer

Initial Implementation and Coarse-fine Search

In this exercise, I used Adam optimizer with the recommended setting $\{\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 1 \times 10^{-8}\}$.

The key modification lies in the update rules implemented within the "MiniBatch" function, where the standard gradient descent updates are replaced by the Adam optimization algorithm. After successfully integrating the Adam optimizer, a coarse-to-fine search is conducted over the regularization strength λ to identify an optimal regularization strength. Here m is set to be 50.

rank	trial_id	lambda	eta_max	p_dropout	best_val_acc
1	22	0.000504	0.071213	0.9	0.551
2	9	1.13E-06	0.045847	0.8	0.545
3	0	5.20E-06	0.088151	0.6	0.544
4	5	0.001025	0.057669	0.8	0.544
5	18	9.39E-05	0.071907	0.8	0.543

Figure 7: Coarse Result

Top 1: Lambda = 2.777213e-04 | Accuracy = 0.5180
Top 2: Lambda = 3.076426e-04 | Accuracy = 0.5138
Top 3: Lambda = 2.567632e-04 | Accuracy = 0.5132

Figure 8: Fine Result

According to the result, the best λ I obtain is 2.78×10^{-4} , and the corresponding test accuracy is 51.80%.

Increase m and Data Augmentation

After finding the optimal λ , I further increase the number of hidden nodes and implement data augmentation(including random horizontal flip and translation). However, the increased number of hidden nodes significantly raises the computational cost. To improve training efficiency, I implement early stop method. The test accuracy is then 52.28%.

Step Decay

What's more, I try step decay strategy to find the appropriate learning rate, with this method, the test accuracy eventually reached **54.30%**. In conclusion, the Adam optimizer demonstrates slightly worse overall performance compared to the SGD with cyclical learning rate approach.

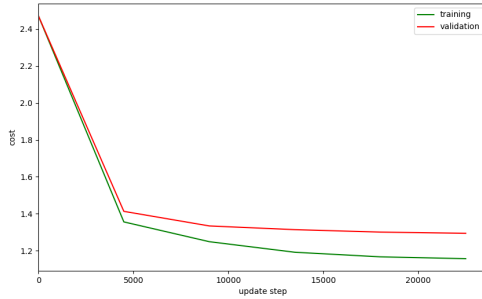


Figure 9: Loss

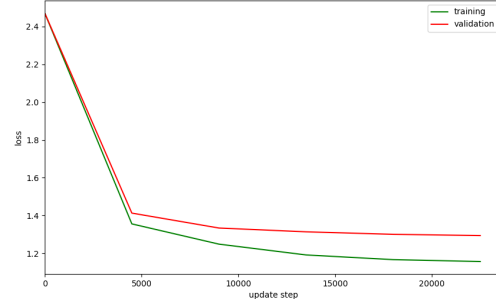


Figure 10: Cost